

Documentação Técnica do Projeto case_datarisk

Credit Scoring e Classificação Preditiva para Concessão de Crédito

Repositório analisado:

/Users/renanscavazzini@gmail.com/Github/case_datarisk

Notebook de documentação:

/Users/renanscavazzini@gmail.com/Github/case_datarisk/docs/documentation

Data de consolidação: 08/06/2026

Idioma: Português

Objetivo do documento

Este notebook consolida a documentação técnica do projeto `case_datarisk`, com foco em modelagem estatística para **credit scoring** e **classificação preditiva de risco**. O conteúdo foi construído com base nos ativos confirmados no repositório, incluindo código-fonte, notebooks analíticos, bases de dados brutas e processadas, artefatos de modelagem e documento técnico complementar.

O objetivo é descrever de forma rastreável:

- o problema de negócio e o contexto analítico;
- a arquitetura do repositório e o fluxo fim a fim;
- as fontes de dados e a lógica de preparação da base;
- a definição da variável alvo `ever_45`;
- as etapas de EDA, engenharia e seleção de atributos;
- a modelagem, validação e escolha do modelo final;
- a política de crédito derivada do score;
- os pontos de deployment, monitoramento, riscos e limitações.

Escopo e critério de evidência

Esta documentação foi baseada nos seguintes ativos já confirmados no projeto:

- `README.md`
- `requirements.txt`
- `runtime.txt`
- módulos em `src/`
- notebooks em `notebooks/01_population` até `07_credit_policy`
- dados em `data/raw/` e `data/processed/`
- artefatos em `outputs/dicts/`, `outputs/models/` e `outputs/submissions/`
- documento `docs/Case Técnico DS.pdf`

Quando um detalhe estiver **claramente suportado** pelos arquivos do repositório, ele será tratado como evidência do projeto. Quando um ponto representar uma **dedução plausível a partir da arquitetura, nomenclatura ou fluxo implementado**, ele será explicitamente descrito como **inferência técnica**. Quando algo relevante **não estiver comprovado** nos ativos analisados, isso será registrado como **ponto não evidenciado**.

Sumário

- Capa e objetivo do projeto
 - Sumário
1. Visão geral da arquitetura do repositório
 - 1.1 Ambiente, dependências e ativos de suporte
 2. Contexto de negócio e problema de credit scoring
 3. Entendimento dos dados
 - 3.1 Fontes de dados brutas confirmadas
 - 3.2 Interpretação funcional das fontes
 - 3.3 Granularidade e chaves
 - 3.4 Papel do dicionário de dados
 - 3.5 Dados processados confirmados
 - 3.6 Granularidade analítica, relacionamento entre bases e cuidados de integração
 - 3.7 Esquemas das tabelas brutas
 - 3.8 Esquemas das tabelas derivadas e bases analíticas
 4. Metodologia fim a fim
 - 4.1 Fluxo operacional entre notebooks, módulos e artefatos
 5. Construção da população
 6. Definição da variável alvo com `target = ever_45`
 - 6.1 Considerações metodológicas sobre a construção do `ever_45`
 7. EDA e principais tratamentos
 - 7.1 Leitura de negócio da EDA
 8. Engenharia de atributos
 - 8.1 Famílias de atributos plausíveis no projeto
 9. Seleção de variáveis
 - 9.1 Seleção de variáveis sob a ótica de robustez
 10. Modelagem e algoritmos comparados
 - 10.1 Racional de comparação entre candidatos
 11. Estratégia de validação com CV, OOS, OOT e métricas
 - 11.1 Importância da validação temporal em crédito
 12. Modelo final LightGBM e critério de escolha por KS em OOT
 13. Política de crédito com ratings A–E e ações associadas
 - 13.1 Uso operacional de ratings na decisão
 14. Geração da submissão
 15. Deployment e operacionalização sugeridos com base nos artefatos existentes
 16. Monitoramento e estabilidade
 17. Riscos, limitações e próximos passos

18. Apêndice: inventário de arquivos e artefatos

19. Conclusão executiva

1. Visão geral da arquitetura do repositório

O repositório `case_datarisk` está organizado de forma compatível com um fluxo de Data Science orientado a produção analítica, separando claramente **código reutilizável**, **notebooks de desenvolvimento**, **dados em diferentes estágios**, **artefatos persistidos** e **documentação**.

Estrutura principal identificada

- `src/` : implementação modular da lógica analítica e de modelagem.
- `notebooks/` : execução encadeada do pipeline analítico em etapas numeradas.
- `data/raw/` : fontes de entrada brutas utilizadas no projeto.
- `data/processed/` : datasets intermediários e finais prontos para treino, score e validação temporal.
- `outputs/dicts/` : dicionários e objetos serializados de apoio ao pipeline.
- `outputs/models/` : modelo final persistido.
- `outputs/submissions/` : arquivo final de submissão do case.
- `docs/` : documentação complementar, incluindo `Case Técnico DS.pdf` e este notebook.
- `README.md` , `requirements.txt` e `runtime.txt` : descrição do projeto e configuração de ambiente.

Leitura arquitetural

A organização sugere um projeto construído com preocupação de **reprodutibilidade** e **separação de responsabilidades**:

- os notebooks representam o fluxo investigativo e executável;
- os módulos em `src/` encapsulam regras e transformações reutilizáveis;
- os dados processados indicam persistência entre etapas para evitar retrabalho;
- os artefatos em `outputs/` mostram que o pipeline avança até a produção de score/modelo/submissão.

Mapeamento entre notebooks e módulos

Há correspondência direta entre os notebooks e os módulos-fonte:

- `01_population` ↔ `src/population.py`
- `02_target_definition` ↔ `src/target.py`
- `03_eda` ↔ `src/eda.py`

- `04_feature_engineering` ↔ `src/feature_engineering.py`
- `05_feature_selection` ↔ `src/feature_selection.py`
- `06_modeling` ↔ `src/modeling.py`
- `07_credit_policy` ↔ `src/policy.py`

Esse desenho evidencia uma evolução típica de projeto: exploração e validação via notebook, seguida de consolidação da lógica em módulos Python.

1.1 Ambiente, dependências e ativos de suporte

Além das etapas analíticas principais, o repositório contém ativos de suporte importantes para reprodutibilidade e manutenção.

Ambiente de execução

- `requirements.txt` : consolida as dependências Python do projeto, incluindo bibliotecas de manipulação de dados, modelagem estatística e machine learning.
- `runtime.txt` : sinaliza o ambiente-base esperado para execução, contribuindo para padronização do runtime.

Módulos utilitários

- `src/data_loader.py` : organiza a carga de dados e serve como referência de padronização documental do projeto.
- `src/utls.py` : concentra funções auxiliares de apoio ao pipeline, reduzindo duplicação de lógica entre notebooks e módulos centrais.

Documentação complementar

- `README.md` : principal fonte de visão geral do projeto, fluxo analítico, organização dos ativos e racional do case.
- `docs/Case Técnico DS.pdf` : documento complementar que reforça a trilha de comunicação técnica do projeto.

Papel desses ativos

Esses componentes não são apenas acessórios. Em conjunto, eles sustentam:

- reprodutibilidade do ambiente;
- padronização da execução;
- reutilização de código;
- rastreabilidade da solução proposta.

2. Contexto de negócio e problema de credit scoring

O projeto está estruturado para resolver um problema de **avaliação de risco de crédito** por meio de um modelo supervisionado de classificação, com saída utilizada para apoiar decisões de concessão e definição de política.

Problema analítico

A lógica central do repositório indica um pipeline para:

- definir uma população elegível de análise;
- construir uma variável alvo de inadimplência;
- gerar atributos comportamentais e cadastrais;
- selecionar variáveis informativas;
- treinar modelos de classificação;
- escolher o modelo com melhor desempenho fora da amostra;
- transformar o score em uma política operacional com ratings e ações.

Leitura do objetivo de negócio

Com base no `README.md`, nos nomes dos notebooks, nos módulos analíticos e na existência de uma submissão final, o objetivo de negócio pode ser descrito como:

estimular uma decisão de crédito mais consistente, mensurável e reproduzível, com base na probabilidade de deterioração futura do cliente.

Natureza do problema

Tecnicamente, trata-se de um problema de:

- **classificação binária;**
- **credit scoring;**
- **priorização/ranqueamento de risco;**
- **apoio à decisão** em política de crédito.

Unidade de decisão

A unidade exata de modelagem deve ser lida em conjunto com a construção de população e a base de submissão. A presença das bases `base_cadastral.parquet`, `base_submissao.parquet` e dos datasets processados `population_score.parquet` e `population_target.parquet` indica que a decisão provavelmente está centrada em uma **entidade elegível para score** (cliente, proposta, contrato ou unidade equivalente definida no case).

Ponto não evidenciado de forma explícita nesta documentação: a granularidade semântica final da entidade decisória não deve ser afirmada além do que estiver diretamente documentado nos notebooks e dicionário. Por isso, esta documentação adota o termo neutro **unidade elegível de score** quando necessário.

3. Entendimento dos dados

3.1 Fontes de dados brutas confirmadas

Foram identificados em `data/raw/` os seguintes arquivos de entrada:

- `base_cadastral.parquet`
- `base_submissao.parquet`
- `historico_emprestimos.parquet`
- `historico_parcelas.parquet`
- `dicionario_dados.csv`

3.2 Interpretação funcional das fontes

Pela nomenclatura dos arquivos, pela existência do dicionário de dados e pela estrutura do pipeline, as fontes cumprem papéis complementares:

- `base_cadastral.parquet` : informações cadastrais e descritivas da unidade elegível;
- `base_submissao.parquet` : base destinada ao score final e geração da submissão;
- `historico_emprestimos.parquet` : histórico transacional/contratual de empréstimos;
- `historico_parcelas.parquet` : histórico de parcelas e comportamento de pagamento;
- `dicionario_dados.csv` : metadados de variáveis, útil para semântica, tipagem e governança analítica.

3.3 Granularidade e chaves

A presença simultânea de base cadastral, histórico de empréstimos e histórico de parcelas sugere um modelo relacional de granularidades distintas:

- uma tabela em nível cadastral ou entidade-base;
- uma tabela em nível de empréstimo/contrato/evento de crédito;
- uma tabela em nível de parcela/pagamento.

Inferência técnica: o pipeline provavelmente consolida atributos agregados do histórico em nível da unidade elegível de score, preservando uma linha por observação modelável na base final.

Ponto não evidenciado explicitamente nesta documentação: os nomes exatos das chaves primárias e estrangeiras não devem ser afirmados sem citar os campos diretamente observados no dicionário ou no código. Portanto, a documentação registra apenas a existência de relacionamento entre níveis cadastral, empréstimo e parcela, sem forçar nomes de identificadores não reproduzidos aqui.

3.4 Papel do dicionário de dados

O arquivo `data/raw/dicionario_dados.csv` é um ativo central do projeto. Sua existência indica preocupação com:

- interpretação semântica das variáveis;
- suporte à análise exploratória;
- padronização de nomes e significados;
- auditoria e rastreabilidade da base analítica.

3.5 Dados processados confirmados

Foram identificados em `data/processed/` os seguintes datasets intermediários e finais:

- `population_active.parquet`
- `population_score.parquet`
- `population_target.parquet`
- `train_base.parquet`
- `train_model.parquet`
- `train_rus_model.parquet`
- `oos_model.parquet`
- `oot_base.parquet`
- `oot_model.parquet`

A nomenclatura sugere segregação explícita entre:

- população ativa e elegível;
- população com target conhecido;
- população destinada a score/submissão;
- bases de treino/modelagem;
- amostras de validação fora da amostra (`OOS`) e fora do tempo (`OOT`).

4. Metodologia fim a fim

O projeto segue uma esteira analítica coerente com boas práticas de modelagem preditiva em risco de crédito.

Etapas macro identificadas

1. Carga e leitura das fontes

Implementada por utilitários de leitura e organização do projeto, com suporte de `src/data_loader.py`.

2. Construção da população

Definição do universo elegível para análise e separação entre população de treino/target e população de score.

3. Definição do alvo

Construção do desfecho supervisionado, explicitamente descrito no pedido como `target = ever_45`.

4. Análise exploratória e saneamento

Diagnóstico de distribuições, ausência, qualidade e comportamento inicial das variáveis.

5. Engenharia de atributos

Criação de variáveis derivadas e agregações históricas com valor preditivo.

6. Seleção de variáveis

Redução de dimensionalidade e retenção de atributos mais relevantes/estáveis.

7. Treinamento e comparação de modelos

Avaliação de abordagens candidatas, com persistência do melhor modelo em `outputs/models/best_model.pkl`.

8. Validação e escolha final

Uso combinado de validação cruzada e janelas de validação externa, com atenção à performance em OOT.

9. Definição de política de crédito

Conversão do score em ratings e ações de negócio.

10. Geração de submissão

Score final aplicado à base de submissão, gerando `outputs/submissions/submissao_case.csv`.

Ativos de implementação

A metodologia está refletida tanto em notebooks exploratórios quanto nos módulos de produção analítica em `src/`, o que reforça a maturidade do pipeline e sua potencial reexecução.

5. Construção da população

A etapa de população está representada pelo notebook `notebooks/01_population` e pelo módulo `src/population.py`.

Objetivo da etapa

A construção da população tem como finalidade definir quais registros entram no universo analítico do projeto, distinguindo pelo menos:

- registros com histórico suficiente para rotulagem e treino;
- registros elegíveis para score futuro/submissão;
- possíveis exclusões por indisponibilidade, inconsistência ou ausência de histórico mínimo.

Evidências observadas no repositório

A existência dos arquivos processados abaixo sustenta a interpretação de que a população foi materializada em estágios:

- `population_active.parquet`
- `population_target.parquet`
- `population_score.parquet`

Interpretação técnica dos datasets

- `population_active.parquet` : indica uma população elegível/ativa após filtros iniciais.
- `population_target.parquet` : indica uma população com possibilidade de construção do alvo supervisionado.
- `population_score.parquet` : indica a população final destinada à inferência e/ou submissão.

Papel metodológico

Em projetos de credit scoring, esta etapa é crítica porque define:

- quem pode ser comparado de forma homogênea no treino;
- quais registros possuem janela de observação suficiente para target;
- quais registros pertencem ao universo de decisão do negócio.

Observação de governança

A separação explícita entre população de target e população de score sugere uma prática importante: **evitar vazamento de informação temporal** entre os conjuntos usados para desenvolvimento e os conjuntos usados para aplicação futura do modelo.

Resultados observados no notebook 01_population

O notebook [01_population](#) executou a construção da população e apresentou os seguintes resultados:

- **Population active:** 81.882 observações (linhas), 28 colunas, 35.353 clientes únicos
- **Population score:** 40.000 observações (linhas), 28 colunas, 40.000 clientes únicos
- **Bases brutas:**
 - base_cadastral: 40.000 clientes
 - base_submissao: 40.000 solicitações para score
 - historico_emprestimos: 186.709 contratos
 - historico_parcelas: 1.407.127 registros
- **Integridade referencial:** 0 clientes sem cadastro, 79.471 contratos sem parcelas (indicando contratos recusados, cancelados ou não efetivados)
- **Cobertura temporal:** histórico disponível cobre aproximadamente 8 anos (2017-02-04 a 2025-02-22)
- **Status dos contratos:** maioria dos contratos encontra-se com status Approved
- **Contratos por cliente:** mediana de 4 contratos por cliente (média ~4.92, máximo 66)
- **Variáveis com ausência significativa:** taxa_juros_padrao, taxa_juros_promocional, data_liberacao

Métricas históricas construídas

O notebook mostra a criação de métricas históricas que incorporam o comportamento passado de cada cliente:

- `qtd_contratos_aceitos_historico` : quantidade histórica de contratos aceitos
- `qtd_contratos_recusados_historico` : quantidade histórica de contratos recusados
- `soma_valor_credito_ativo_historico` : soma histórica de crédito ativo
- `media_valor_credito_aceito_historico` : média histórica de crédito aceito

Essas variáveis representam agregações do histórico de cada cliente até a data de observação, sem vazamento de informação futura.

6. Definição da variável alvo com `target = ever_45`

A etapa de definição de target está representada pelo notebook `notebooks/02_target_definition` e pelo módulo `src/target.py`.

Definição conceitual

O projeto explicita que a variável alvo é baseada em `ever_45`. Em contexto de risco de crédito, essa nomenclatura é usualmente interpretada como um indicador binário de

ocorrência de atraso igual ou superior a **45 dias** em algum momento da janela de performance.

Interpretação adotada nesta documentação

Assim, o alvo pode ser descrito como:

target = 1 quando a unidade observacional apresenta evento de atraso compatível com a regra `ever_45`; caso contrário, `target = 0`.

Observações de evidência

- A expressão `ever_45` foi explicitamente solicitada pelo usuário e está alinhada com a nomenclatura típica do domínio.
- A materialização de `population_target.parquet` reforça que houve uma etapa dedicada à rotulagem supervisionada.
- A existência de `historico_parcelas.parquet` e `historico_emprestimos.parquet` indica que o alvo provavelmente é derivado de comportamento histórico de pagamento.

Cuidados metodológicos esperados

Uma definição adequada de target em risco exige, no mínimo:

- delimitação clara da janela de observação;
- delimitação clara da janela de performance;
- exclusão de informações futuras em relação ao ponto de decisão;
- coerência entre a população elegível e a disponibilidade histórica.

Resultados observados no notebook 02_target_definition

O notebook [02_target_definition](#) executou a construção do target e apresentou os seguintes resultados:

3.1 Comparação de definições candidatas

Foram comparadas quatro definições de target (`ever_30`, `ever_45`, `ever_60`, `ever_90`) para avaliar diferentes níveis de severidade de inadimplência:

definition	total	good	bad	% bad
ever_30	81.882	80.798	1.084	1.32%
ever_45	81.882	81.362	520	0.63%
ever_60	81.882	81.548	334	0.41%

definition	total	good	bad	% bad
ever_90	81.882	81.676	206	0.25%

Justificativa da escolha: o projeto adotou `ever_45` como target final, equilibrando (1) representação de um evento de inadimplência suficientemente severo para refletir risco de crédito real e (2) volume adequado de eventos para permitir treinamento estatisticamente robusto.

3.2 Divisão temporal das bases

Períodos de treino e OOT foram explicitamente definidos:

- **TRAIN_PERIODS:** de 2020-01 até 2023-12 (48 meses)
- **OOT_PERIODS:** de 2024-01 até 2025-01 (13 meses)

Volumetria final:

- **Train base:** 61.912 observações, 34 colunas
- **OOT base:** 19.970 observações, 34 colunas

3.3 Distribuição do target por safra

Train: taxa de inadimplência (% bad) variou entre 0.14% e 3.81% ao longo das safras mensais, com taxa total de **0.82%**.

OOT: taxa de inadimplência variou entre 0.00% e 0.19% ao longo das safras mensais, com taxa total de **0.06%**.

Interpretação técnica: a redução da taxa de inadimplência na base OOT sugere que os contratos mais recentes ainda não tiveram tempo de maturar suficientemente para manifestar atraso, ou que houve melhoria genuína no perfil de risco da carteira. Essa característica é esperada em validação temporal em bases de crédito, e deve ser considerada na interpretação das métricas de performance OOT.

Ponto não evidenciado nesta documentação

Detalhes adicionais de parametrização temporal do `ever_45` além do que está documentado acima — por exemplo, regras de cure ou consolidação de múltiplos contratos — não devem ser afirmados numericamente aqui sem reprodução textual dos critérios diretamente a partir do notebook ou módulo. Onde necessário, esta documentação trata essa definição como **regra de inadimplência baseada em atraso de 45+ dias**.

3.6 Granularidade analítica, relacionamento entre bases e cuidados de integração

A leitura conjunta dos ativos aponta para uma arquitetura de dados multigranular, típica de crédito.

Níveis analíticos prováveis

- nível cadastral da entidade elegível;
- nível histórico de operações/empréstimos;
- nível histórico de parcelas e eventos de pagamento.

Implicações técnicas

A consolidação dessas fontes exige cuidados com:

- deduplicação antes de agregações;
- definição do ponto de observação temporal;
- prevenção de vazamento de informação;
- agregação consistente para uma única linha por observação modelável.

Papel das bases processadas

As bases processadas sugerem um encadeamento de integração e enriquecimento:

- `population_active.parquet` e `population_target.parquet` representam populações em estágios distintos do processo de elegibilidade e rotulagem;
- `train_base.parquet` e `oot_base.parquet` indicam bases analíticas ainda anteriores ao recorte final de modelagem;
- `train_model.parquet`, `oos_model.parquet` e `oot_model.parquet` indicam conjuntos finalizados para treino e avaliação.

Cuidado documental

Sem reproduzir esquemas completos no corpo desta documentação, a interpretação acima deve ser lida como **inferência técnica fortemente suportada pela nomenclatura dos artefatos** e pelo fluxo modular do repositório.

3.7 Esquemas das tabelas brutas

Esta seção consolida os **esquemas lógicos** das tabelas de entrada do projeto com base, principalmente, em `data/raw/dicionario_dados.csv`.

Critério de leitura

- As colunas e descrições abaixo estão **evidenciadas** no dicionário de dados do repositório.
- O campo **tipo semântico** foi incluído como apoio documental e deve ser lido como **interpretação técnica**, não como dtype físico confirmado do arquivo parquet.

base_cadastral

Coluna	Tipo semântico	Descrição
id_cliente	chave	Identificador único do cliente
sexo	categórica	Sexo do cliente (M ou F)
data_nascimento	data	Data de nascimento do cliente
qtd_filhos	numérica discreta	Quantidade de filhos declarados
qtd_membros_familia	numérica discreta	Quantidade total de membros da família
renda_anual	numérica contínua	Renda total anual declarada pelo cliente
tipo_renda	categórica	Tipo de fonte de renda
ocupacao	categórica	Ocupação profissional do cliente
tipo_organizacao	categórica	Tipo de empresa onde o cliente trabalha
nivel_educacao	categórica	Nível de escolaridade alcançado
estado_civil	categórica	Estado civil do cliente
tipo_moradia	categórica	Tipo de moradia
possui_carro	flag/categórica	Indicador se o cliente possui carro (Y/N)
possui_imovel	flag/categórica	Indicador se o cliente possui imóvel (Y/N)
nota_regiao_cliente	numérica	Avaliação de risco da região de residência do cliente
nota_regiao_cliente_cidade	numérica	Avaliação de risco da região ajustada para a cidade

base_submissao

Coluna	Tipo semântico	Descrição
id_cliente	chave	Identificador único do cliente
data_solicitacao	data	Data em que o contrato mais recente foi solicitado

Coluna	Tipo semântico	Descrição
dia_semana_solicitacao	categórica/temporal	Dia da semana em que o pedido foi iniciado
hora_solicitacao	categórica/temporal	Hora do dia em que a solicitação foi realizada
tipo_contrato	categórica	Tipo de contrato solicitado
valor_credito	numérica contínua	Valor total do crédito solicitado
valor_bem	numérica contínua	Valor do bem financiado, quando aplicável
valor_parcela	numérica contínua	Valor estimado da parcela periódica

historico_emprestimos

Coluna	Tipo semântico	Descrição
id_contrato	chave	Identificador único do contrato de empréstimo
id_cliente	chave relacional	Identificador do cliente associado ao contrato
tipo_contrato	categórica	Tipo de contrato
status_contrato	categórica	Status final do contrato
data_decisao	data	Data em que o contrato foi analisado e aprovado ou recusado
data_liberacao	data	Data em que o crédito foi liberado
data_primeiro_vencimento	data	Data prevista para o vencimento da primeira parcela
data_ultimo_vencimento_original	data	Data original prevista para a última parcela
data_ultimo_vencimento	data	Data final revisada da última parcela
data_encerramento	data	Data em que o contrato foi encerrado
valor_solicitado	numérica contínua	Valor inicialmente solicitado pelo cliente
valor_credito	numérica contínua	Valor efetivamente liberado
valor_bem	numérica contínua	Valor do bem adquirido, quando aplicável

Coluna	Tipo semântico	Descrição
valor_parcela	numérica contínua	Valor estimado da parcela periódica
valor_entrada	numérica contínua	Valor pago como entrada
percentual_entrada	numérica contínua	Percentual da entrada sobre o valor total
qtd_parcelas_planejadas	numérica discreta	Quantidade de parcelas previstas no contrato
taxa_juros_padrao	numérica contínua	Taxa de juros padrão aplicada
taxa_juros_promocional	numérica contínua	Taxa promocional, se aplicável
tipo_pagamento	categórica	Forma de pagamento do contrato
finalidade_emprestimo	categórica	Finalidade declarada para uso do empréstimo
tipo_cliente	categórica	Tipo de cliente
faixa_rendimento	categórica	Categoria de retorno do cliente para o banco
tipo_portfolio	categórica	Tipo de portfólio
tipo_produto	categórica	Tipo de produto financeiro contratado
categoria_bem	categórica	Categoria do bem financiado
combinacao_produto	categórica	Combinação de produtos oferecida
setor_vendedor	categórica	Setor da empresa vendedora
canal_venda	categórica	Canal utilizado para origemação da venda
area_venda	categórica	Área ou localização da venda
dia_semana_solicitacao	categórica/temporal	Dia da semana em que a solicitação foi iniciada
hora_solicitacao	categórica/temporal	Hora do dia em que a solicitação foi iniciada
flag_ultima_solicitacao_contrato	flag	Indica se foi a última solicitação para aquele contrato
flag_ultima_solicitacao_dia	flag	Indica se foi a última solicitação feita no mesmo dia

Coluna	Tipo semântico	Descrição
<code>motivo_recusa</code>	categórica	Motivo da recusa, quando o contrato foi negado
<code>acompanhantes_cliente</code>	categórica	Tipo de acompanhantes no momento da solicitação
<code>flag_seguro_contratado</code>	flag	Indica se o cliente contratou seguro junto com o crédito

historico_parcelas

Coluna	Tipo semântico	Descrição
<code>id_contrato</code>	chave relacional	Identificador do contrato de empréstimo
<code>id_cliente</code>	chave relacional	Identificador do cliente
<code>versao_parcela</code>	numérica discreta	Versão da parcela dentro do contrato
<code>numero_parcela</code>	numérica discreta	Número sequencial da parcela dentro do contrato
<code>data_prevista_pagamento</code>	data	Data inicialmente prevista para o pagamento da parcela
<code>data_real_pagamento</code>	data	Data real em que o pagamento foi efetuado
<code>valor_previsto_parcela</code>	numérica contínua	Valor original previsto da parcela
<code>valor_pago_parcela</code>	numérica contínua	Valor efetivamente pago pelo cliente

3.8 Esquemas das tabelas derivadas e bases analíticas

Esta seção registra os esquemas das principais bases derivadas **comprovadas pelo código-fonte** e pela nomenclatura dos artefatos processados.

Fontes de evidência utilizadas

- `src/population.py`
- `src/target.py`
- inventário dos arquivos em `data/processed/`

population_active.parquet e population_score.parquet

O módulo `src/population.py` define um esquema final padronizado para as populações **ativa** e **de score**. As colunas são:

Coluna	Papel semântico
<code>id_cliente</code>	Identificador do cliente
<code>data_solicitacao</code>	Data de referência da observação
<code>dia_semana_solicitacao_submissao</code>	Dia da semana da solicitação
<code>hora_solicitacao_submissao</code>	Hora da solicitação
<code>tipo_contrato_submissao</code>	Tipo de contrato solicitado
<code>valor_credito_submissao</code>	Valor do crédito solicitado
<code>valor_bem_submissao</code>	Valor do bem associado
<code>valor_parcela_submissao</code>	Valor estimado da parcela
<code>sexo</code>	Sexo do cliente
<code>data_nascimento</code>	Data de nascimento
<code>qtd_filhos</code>	Quantidade de filhos
<code>qtd_membros_familia</code>	Quantidade de membros da família
<code>renda_anual</code>	Renda anual declarada
<code>tipo_renda</code>	Tipo de renda
<code>ocupacao</code>	Ocupação
<code>tipo_organizacao</code>	Tipo de organização
<code>nivel_educacao</code>	Escolaridade
<code>estado_civil</code>	Estado civil
<code>tipo_moradia</code>	Tipo de moradia
<code>possui_carro</code>	Indicador de posse de carro
<code>possui_imovel</code>	Indicador de posse de imóvel
<code>nota_regiao_cliente</code>	Nota da região do cliente
<code>nota_regiao_cliente_cidade</code>	Nota da região ajustada para a cidade
<code>safra_mes</code>	Safra mensal da observação
<code>qtd_contratos_aceitos_historico</code>	Quantidade histórica de contratos aceitos
<code>qtd_contratos_recusados_historico</code>	Quantidade histórica de contratos recusados
<code>soma_valor_credito_ativo_historico</code>	Soma histórica de crédito ativo/aceito

Coluna	Papel semântico
<code>media_valor_credito_aceito_historico</code>	Média histórica de crédito aceito

population_target.parquet

O módulo `src/target.py` evidencia a construção de uma base de target com as colunas abaixo:

Coluna	Papel semântico
<code>id_cliente</code>	Identificador do cliente
<code>safra_mes</code>	Safra da observação
<code>id_contrato</code>	Contrato representativo associado à observação
<code>max_delay</code>	Maior atraso observado no contrato
<code>ever_30</code>	Flag de atraso acima de 30 dias
<code>ever_45</code>	Flag de atraso acima de 45 dias
<code>ever_60</code>	Flag de atraso acima de 60 dias
<code>ever_90</code>	Flag de atraso acima de 90 dias
<code>target</code>	Variável alvo binária derivada de <code>ever_45</code>

Demais bases processadas

Os seguintes arquivos existem em `data/processed/`, mas seus esquemas completos não foram reproduzidos diretamente nesta documentação sem leitura física dos parquet:

- `train_base.parquet`
- `train_model.parquet`
- `train_rus_model.parquet`
- `oos_model.parquet`
- `oot_base.parquet`
- `oot_model.parquet`

Interpretação técnica segura

Pela posição dessas bases no pipeline, elas representam recortes sucessivos da base analítica para treino, balanceamento, validação fora da amostra e validação temporal.

Ponto não evidenciado integralmente

Os **schemas físicos completos** dessas bases derivadas não foram afirmados linha a linha aqui para evitar inferir colunas além do que está explicitamente comprovado no código-fonte já consolidado.

4.1 Fluxo operacional entre notebooks, módulos e artefatos

O projeto mostra um desenho em que o desenvolvimento ocorreu de forma progressiva e estruturada.

Camada 1: exploração e validação por notebook

Os notebooks numerados funcionam como trilha narrativa e executável do projeto, permitindo desenvolver, testar e apresentar cada etapa do pipeline.

Camada 2: consolidação em módulos Python

Os módulos em `src/` encapsulam a lógica que tende a ser mais estável, reutilizável e apropriada para reexecução. Esse padrão reduz acoplamento com células exploratórias e melhora a manutenção.

Camada 3: persistência de saídas intermediárias

A gravação de datasets em `data/processed/` indica estratégia de checkpoint analítico. Isso permite:

- reuso entre etapas;
- economia de tempo computacional;
- inspeção intermediária das bases;
- facilitação de debug e auditoria.

Camada 4: persistência de artefatos de modelagem

Os diretórios `outputs/dicts/` e `outputs/models/` mostram que o pipeline persiste não apenas o modelo final, mas também estruturas auxiliares necessárias para consistência entre treino e aplicação.

Camada 5: entrega final

A presença de `outputs/submissions/submissao_case.csv` demonstra que o fluxo foi concluído até a etapa de entrega do resultado final do case.

6.1 Considerações metodológicas sobre a construção do `ever_45`

Em projetos de crédito, a definição do target é uma das decisões mais críticas para a validade do modelo.

Elementos que normalmente precisam estar alinhados

- data de referência da observação;
- janela histórica disponível para construção de atributos;
- janela futura usada para observação do evento de atraso;
- regra de consolidação quando há múltiplos contratos ou múltiplas parcelas;
- tratamento de casos sem maturação suficiente.

Por que isso importa

A má definição do target pode produzir:

- vazamento de informação;
- estimativa inflada de performance;
- política de crédito inconsistente;
- instabilidade quando o modelo é levado para produção.

Leitura para este repositório

A existência de um notebook exclusivo para target e de datasets separados para população/target é um sinal positivo de governança metodológica, pois sugere que a rotulagem não foi tratada como detalhe secundário, e sim como etapa formal do pipeline.

7. EDA e principais tratamentos

A análise exploratória está associada ao notebook `notebooks/03_eda` e ao módulo `src/eda.py`.

Objetivos da EDA no contexto do projeto

A EDA, em um pipeline de credit scoring como este, cumpre os seguintes papéis:

- entender distribuição das variáveis e do target;
- avaliar qualidade e completude das bases;
- identificar variáveis inconsistentes, redundantes ou pouco informativas;
- orientar regras de limpeza e transformações posteriores;
- apoiar a leitura de estabilidade entre amostras e janelas.

Tratamentos esperados e compatíveis com o repositório

Considerando a estrutura dos módulos e dos datasets processados, os principais tratamentos realizados ou preparados para o pipeline incluem:

- padronização de tipos e leitura de dados;
- tratamento de ausências;
- análise de cardinalidade e categorias raras, quando aplicável;
- inspeção de variáveis numéricas com potenciais extremos;
- construção de visões agregadas para compreender comportamento histórico.

Papel do dicionário na EDA

O arquivo `dicionario_dados.csv` fornece suporte para:

- interpretar colunas sem ambiguidade;
- separar variáveis cadastrais, históricas e possivelmente derivadas;
- documentar variáveis candidatas à modelagem.

Limite factual

Esta documentação reconhece que houve uma etapa estruturada de EDA, mas evita afirmar resultados numéricos não reproduzidos diretamente nesta consolidação.

7.1 Leitura de negócio da EDA

Além do papel estatístico, a EDA em um projeto de score de crédito deve responder perguntas de negócio como:

- quem é a população analisada;
- quais atributos parecem melhor discriminar risco;
- onde existem lacunas de informação;
- quais variáveis podem comprometer robustez por instabilidade ou baixa qualidade.

Resultado esperado da etapa

Mesmo sem reproduzir tabelas e gráficos neste notebook, a etapa de EDA deve ter servido para orientar:

- exclusões e limpezas;
- priorização de famílias de variáveis;
- desenho da engenharia de atributos;
- escolhas de validação e monitoramento posteriores.

Relação com o caso técnico

A existência do documento `docs/Case Técnico DS.pdf` sugere que parte dos achados exploratórios e narrativas analíticas também pode ter sido comunicada em formato executivo ou técnico fora do código-fonte.

8. Engenharia de atributos

A etapa está representada pelo notebook `notebooks/04_feature_engineering` e pelo módulo `src/feature_engineering.py`.

Objetivo

Transformar dados brutos cadastrais e históricos em uma base analítica com maior poder preditivo para o problema de inadimplência.

Direções de engenharia inferidas a partir das fontes

Dada a disponibilidade de histórico de empréstimos e parcelas, a engenharia de atributos provavelmente contempla famílias como:

- agregações históricas por unidade elegível;
- contagens de eventos de crédito;
- indicadores de atraso e regularidade de pagamento;
- estatísticas de intensidade, frequência e recorrência;
- combinações entre atributos cadastrais e comportamentais.

Materialização no pipeline

A presença de bases como `train_base.parquet` e `train_model.parquet` indica que houve ao menos duas camadas distintas:

- uma base preparada para análise e modelagem;
- uma base já filtrada ou transformada para treino efetivo.

Boas práticas refletidas na arquitetura

A separação da engenharia em módulo próprio sugere:

- reuso das transformações entre treino e aplicação;
- redução de divergência entre notebook exploratório e pipeline final;
- possibilidade de serialização de objetos auxiliares em `outputs/dicts/`.

8.1 Famílias de atributos plausíveis no projeto

Com base nas fontes disponíveis, é tecnicamente plausível que a engenharia de atributos tenha explorado grupos como:

- atributos cadastrais brutos ou recodificados;
- agregações históricas de quantidade de empréstimos;
- medidas de recorrência de atraso;
- estatísticas de parcelas pagas, vencidas ou em atraso;
- indicadores temporais de comportamento recente versus histórico;
- razões e proporções derivadas do relacionamento entre eventos.

Cuidados usuais nessa etapa

- congelar a janela de observação antes da performance;
- tratar cardinalidade elevada em variáveis categóricas;
- evitar atributos com forte risco de leakage;
- preservar consistência entre treino, validação e score.

Relação com artefatos persistidos

Os objetos em `outputs/dicts/*.pkl` podem suportar listas de colunas, encodings, agrupamentos, thresholds ou dicionários auxiliares usados nesta etapa. Como o conteúdo exato dos arquivos não foi reproduzido aqui, isso é tratado como inferência técnica aderente aos artefatos existentes.

9. Seleção de variáveis

A etapa está associada ao notebook `notebooks/05_feature_selection` e ao módulo `src/feature_selection.py`.

Papel no projeto

A seleção de variáveis é especialmente importante em credit scoring para equilibrar:

- desempenho preditivo;
- robustez fora da amostra;
- estabilidade temporal;
- interpretabilidade operacional;
- redução de ruído e redundância.

Evidências do pipeline

A existência dos datasets abaixo sugere refinamento progressivo da base:

- `train_base.parquet`
- `train_model.parquet`
- `train_rus_model.parquet`
- `oos_model.parquet`
- `oot_model.parquet`

Essa nomenclatura é compatível com uma trilha em que a seleção de variáveis atua antes do treino final e também sustenta diferentes amostras de avaliação.

Possíveis critérios utilizados

Sem afirmar regras não reproduzidas numericamente, os critérios tecnicamente esperados para esta etapa incluem:

- remoção de variáveis com alto missing ou baixa variância;
- tratamento de colinearidade e redundância;
- priorização por importância preditiva;
- avaliação de estabilidade e generalização;
- compatibilização com restrições de negócio e política.

Observação sobre `train_rus_model.parquet`

A presença desse dataset sugere o uso de alguma estratégia de balanceamento por random undersampling.

9.1 Seleção de variáveis sob a ótica de robustez

Em crédito, a seleção de variáveis não deve focar apenas em ganho marginal de performance em treino.

Critérios desejáveis

- estabilidade entre amostras;
- coerência temporal;
- capacidade de generalização;
- aderência à política de uso do score;
- facilidade de manutenção das variáveis em produção.

Sinalizações do repositório

A existência de OOS e OOT como conjuntos nomeados e a escolha final por KS em OOT sugerem que a robustez fora da amostra foi mais importante do que maximizar apenas métricas internas. Isso também afeta, direta ou indiretamente, a seleção das variáveis mantidas no pipeline final.

10. Modelagem e algoritmos comparados

A etapa está representada pelo notebook `notebooks/06_modeling` e pelo módulo `src/modeling.py`.

Objetivo

Treinar e comparar algoritmos de classificação para estimar o risco associado à variável alvo `ever_45`, priorizando capacidade discriminatória e estabilidade em validação externa.

Resultados observados no notebook 06_modeling

O notebook executou treinamento e comparação de três algoritmos.

10.1 Volumetria das bases de modelagem

- **train_model:** 49.529 observações, 27 colunas
- **train_rus_model:** 814 observações, 27 colunas
- **oos_model:** 12.383 observações, 27 colunas
- **oot_model:** 19.970 observações, 27 colunas

10.2 Conjunto de features selecionadas

- **Modelos lineares:** 14 variáveis
- **Modelos baseados em árvore:** 24 variáveis

10.3 Algoritmos treinados e métricas OOT

Modelo	AUC (OOT)	KS (OOT)	Gini (OOT)	PSI (OOT)
Logistic Regression	0.5775	0.2916	0.1550	0.1737
Random Forest	0.6361	0.2688	0.2721	0.2260
LightGBM	0.6211	0.3316	0.2421	0.1554

O LightGBM foi escolhido como modelo final por equilibrar melhor poder discriminatório e estabilidade temporal.

10.1 Racional de comparação entre candidatos

A etapa de modelagem, neste tipo de projeto, normalmente busca equilibrar:

- performance preditiva;
- estabilidade em OOT;
- simplicidade operacional;
- coerência com a política de crédito;
- capacidade de reaplicação na base de submissão.

Leitura documental segura

É seguro afirmar que houve processo comparativo entre candidatos, pois há uma etapa dedicada à modelagem, um artefato `best_model.pkl` e um critério final explicitado. O que não deve ser afirmado sem evidência adicional é a lista fechada de todos os challengers e respectivos hiperparâmetros além do que já está documentado.

11. Estratégia de validação com CV, OOS, OOT e métricas

Estrutura de validação evidenciada

O projeto apresenta uma estratégia de validação compatível com contexto de risco de crédito e generalização temporal, apoiada pelos datasets:

- `train_model.parquet`
- `oos_model.parquet`
- `oot_model.parquet`

Além disso, o projeto explicita a utilização de:

- **CV** (cross-validation)
- **OOS** (out-of-sample)
- **OOT** (out-of-time)
- métricas **AUC**, **KS**, **Gini** e **PSI**

Leitura metodológica

A combinação dessas frentes permite avaliar dimensões diferentes do modelo:

- **CV**: robustez interna no conjunto de desenvolvimento;
- **OOS**: generalização em amostra separada da etapa de treino;
- **OOT**: estabilidade e performance em recorte temporal posterior;

- **PSI**: monitoramento de mudança de distribuição e estabilidade populacional.

11.1 Importância da validação temporal em crédito

A separação OOT merece destaque específico.

Por que OOT é central

Modelos de risco de crédito são aplicados prospectivamente. Por isso, validar em recorte temporal posterior é uma forma mais realista de estimar desempenho sob mudança de contexto.

Benefícios da abordagem

- melhor aproximação do ambiente de produção;
- menor risco de superestimar qualidade do modelo;
- maior aderência entre modelagem e política de crédito;
- suporte à avaliação de estabilidade do score ao longo do tempo.

Relação com PSI

A inclusão de PSI na documentação metodológica é coerente com essa preocupação, pois complementa a validação discriminatória com análise de deslocamento de distribuição.

12. Modelo final LightGBM e critério de escolha por KS em OOT

Modelo escolhido

O modelo final documentado para o projeto é um **LightGBM**, persistido em:

- `outputs/models/best_model.pkl`

Critério de escolha

Conforme solicitado e alinhado com a estrutura do projeto, o critério de decisão do modelo final foi o desempenho por **KS em OOT**.

Interpretação técnica do critério

Escolher o modelo com base em KS na base out-of-time é metodologicamente consistente em crédito porque:

- prioriza estabilidade temporal;
- aproxima a avaliação do cenário real de uso do modelo;
- reduz o risco de selecionar modelos com sobreajuste em validações internas;
- valoriza a capacidade de separação em janela futura, mais relevante para política.

13. Política de crédito com ratings A–E e ações associadas

A etapa de política está representada pelo notebook `notebooks/07_credit_policy` e pelo módulo `src/policy.py`.

Objetivo

Traduzir a saída do modelo em uma regra operacional de decisão, segmentando a população por faixas de risco e associando ações de negócio.

Estrutura documentada

A política foi organizada em ratings **A, B, C, D e E**, com ações associadas.

Interpretação funcional

Em um fluxo de credit scoring, essa etapa converte a probabilidade do modelo em:

- bandas ordenadas de risco;
- thresholds de decisão;
- ação recomendada por banda.

13.1 Uso operacional de ratings na decisão

A transformação do score em ratings A–E representa a ponte entre modelo estatístico e decisão de negócio.

Benefícios da abordagem por bandas

- simplifica comunicação com áreas de crédito;
- facilita definição de alçadas e estratégias;
- permite monitorar performance por faixa de risco;

- apoia revisão de política sem necessidade de alterar o motor estatístico em toda mudança operacional.

Estrutura recomendada de governança

Para produção, seria desejável versionar explicitamente:

- cortes de score por rating;
- ação correspondente por banda;
- data de vigência da política;
- racional de negócio para cada alteração.

14. Geração da submissão

O repositório contém o artefato final de submissão:

- `outputs/submissions/submissao_case.csv`

Leitura do fluxo final

A geração da submissão pressupõe, no mínimo:

- construção da população de score;
- aplicação das mesmas transformações necessárias ao modelo final;
- inferência com o `best_model.pkl` ;
- formatação da saída conforme o padrão exigido pelo case.

Resultados observados no notebook 07_credit_policy

O notebook executou a escoragem da base de submissão e gerou o arquivo final de saída para o universo de 40.000 clientes.

15. Deployment e operacionalização sugeridos com base nos artefatos existentes

O repositório não apresenta, nesta consolidação, uma camada explícita de serving online, API ou orquestração produtiva. Ainda assim, os artefatos existentes permitem desenhar uma proposta realista de operacionalização.

Ativos já existentes que apoiam operacionalização

- módulos reutilizáveis em `src/` ;
- datasets processados persistidos em `data/processed/` ;
- objetos auxiliares serializados em `outputs/dicts/` ;
- modelo final em `outputs/models/best_model.pkl` ;
- política de crédito formalizada em notebook ou módulo dedicado.

16. Monitoramento e estabilidade

Embora o repositório analisado seja predominantemente voltado ao desenvolvimento analítico, a própria metodologia documentada indica os principais eixos de monitoramento necessários após implantação.

Dimensões recomendadas

1. Estabilidade populacional

Monitorar mudanças de distribuição entre a base de desenvolvimento e a base corrente, com uso de **PSI** em variáveis-chave, score e bandas de política.

2. Performance discriminatória

Acompanhar periodicamente **AUC**, **KS** e **Gini** nas coortes em que o target já maturou.

17. Riscos, limitações e próximos passos

Riscos e limitações observados

1. Limitação de evidência numérica nesta consolidação

Esta documentação foi construída com base nos ativos confirmados do repositório, mas sem reproduzir integralmente todas as saídas analíticas dos notebooks.

2. Dependência de coerência temporal

Projetos com target do tipo `ever_45` exigem rigor em janelas de observação e performance. Qualquer desalinhamento temporal pode gerar vazamento de informação.

3. Sensibilidade a drift

Modelos de crédito são particularmente sensíveis a mudanças de comportamento populacional, conjuntura econômica e alterações na política de originação.

18. Apêndice: inventário de arquivos e artefatos

18.1 Arquivos de configuração e documentação

- README.md
- requirements.txt
- requirements-dev.txt
- runtime.txt
- .gitignore
- docs/Case Técnico DS.pdf
- docs/documentation (este notebook)

18.2 Módulos fonte em src/

- data_loader.py
- population.py
- target.py
- eda.py
- feature_engineering.py
- feature_selection.py
- modeling.py
- policy.py
- utils.py

19. Conclusão executiva

O repositório `case_datarisk` evidencia uma solução de Data Science estruturada para um problema de **credit scoring supervisionado**, com pipeline modular e rastreável desde a preparação da população até a geração da submissão final.

Síntese do que está comprovado

- há fontes brutas, dados processados e dicionário de dados;
- o pipeline está segmentado em etapas clássicas de modelagem de risco;
- os notebooks e módulos cobrem população, target, EDA, engenharia, seleção, modelagem e política;
- existe modelo final persistido em LightGBM;
- há artefato final de submissão.

Avaliação geral

Sob a ótica de documentação técnica, o projeto apresenta maturidade compatível com um case robusto de risco de crédito, com boa separação entre desenvolvimento analítico, modularização de código, persistência de artefatos e possibilidade de evolução para uma rotina operacional controlada.